

Бинарная Сортировка Вставкой

Binary Insertion Sort

Что такое сортировка вставкой?

В простом алгоритме сортировки вставкой мы поддерживаем отсортированный и несортированный подмассив заданного массива. В каждой итерации выбирается одно значение из несортированной части и вставляется в его правильную позицию в отсортированной части. Чтобы добиться этого, для каждого элемента мы перебираем отсортированную часть массива, чтобы найти позицию для вставки элемента.

Для вставки одного элемента требуется:

- $O(N)$ сравнений

- $O(N)$ перестановок

Бинарный Поиск

Бинарный поиск осуществляется только в отсортированном списке.

1

2

3

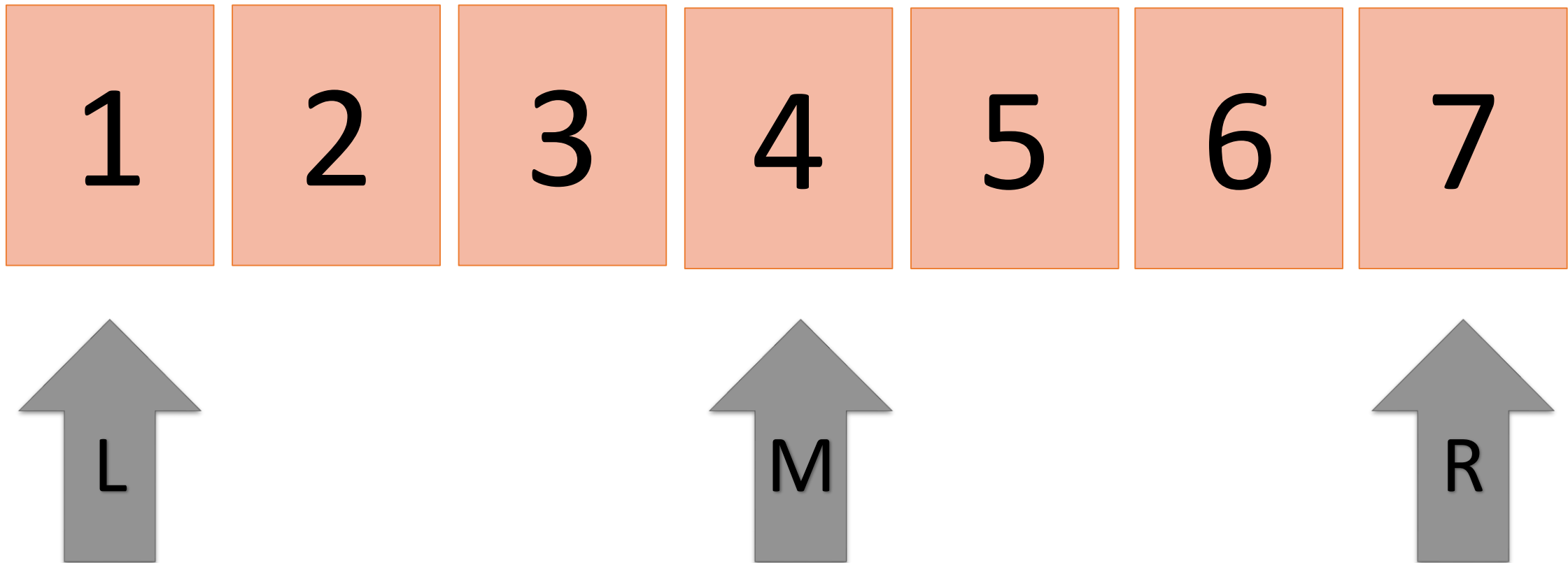
4

5

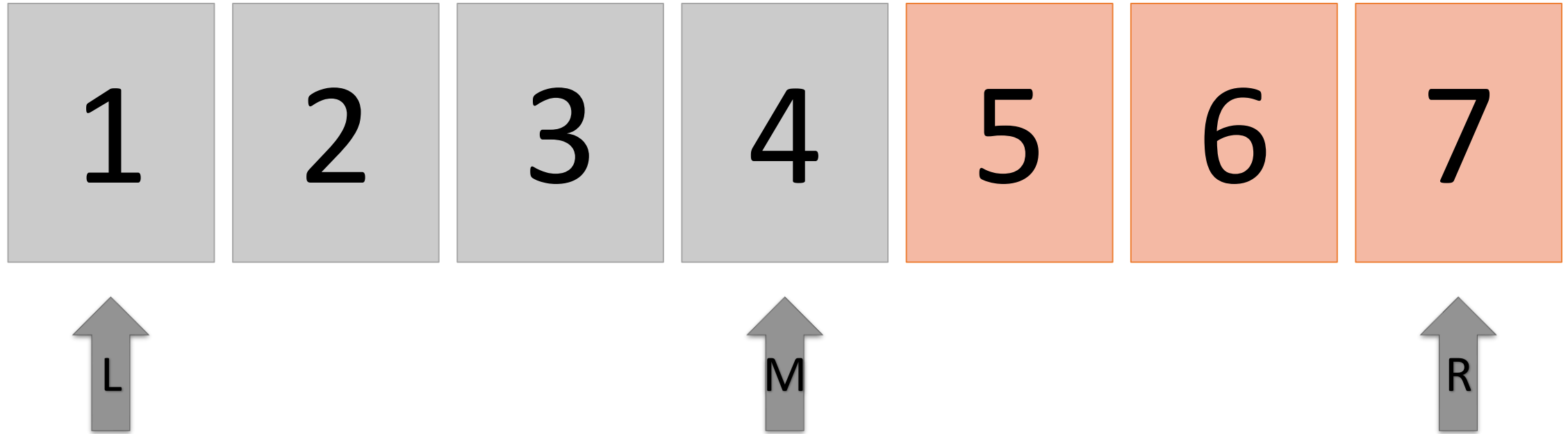
6

Бинарный Поиск

Если мы хотим найти 5, то мы находим левый правый и средний элементы.

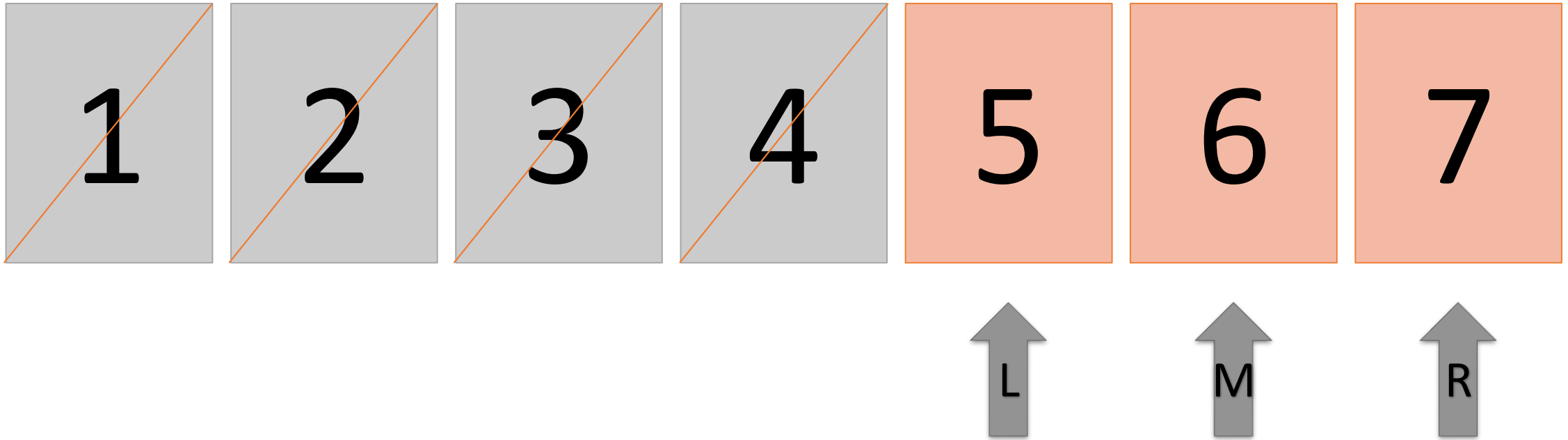


Бинарный Поиск



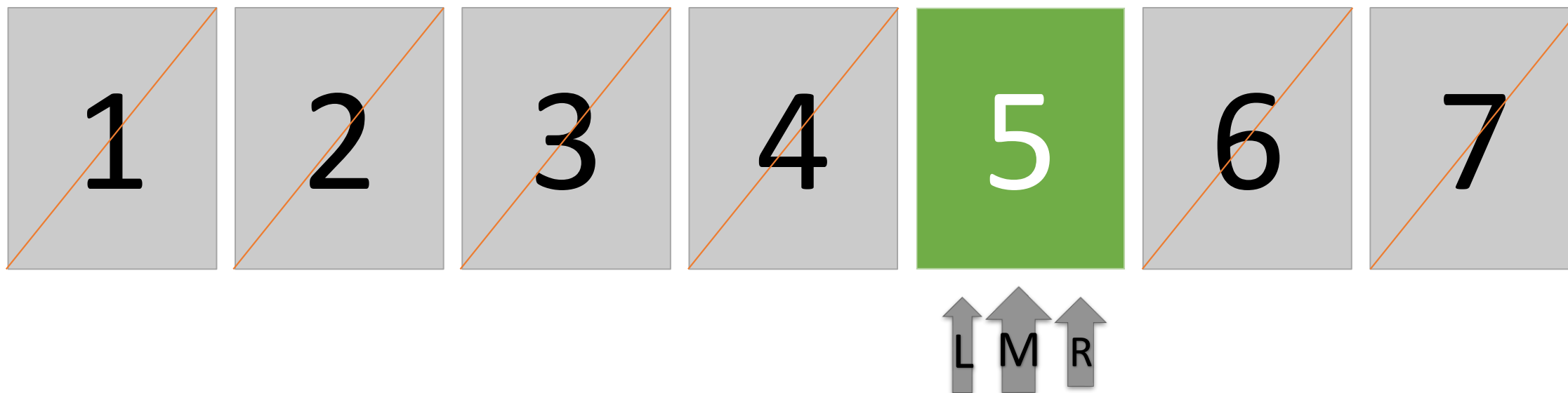
Мы исключаем всю правую сторону.

Бинарный Поиск



Мы исключаем всю правую сторону.

Бинарный Поиск



Мы исключаем 6 и 7.

Бинарная Сортировка Вставкой

Объединяя эти две идеи, мы выполняем нашу бинарную сортировку вставкой.

Бинарный Поиск+ Сортировка Вставкой =
Бинарная Сортировка Вставкой

Что такое бинарная сортировка вставкой?

Это алгоритм сортировки, похожий на сортировку вставкой, но вместо использования линейного поиска для нахождения позиции, куда следует вставить элемент, мы используем бинарный поиск.

Таким образом, мы сокращаем количество сравнений для вставки одного элемента с:

$$O(N) \rightarrow O(\log N).$$

Как работает бинарная сортировка вставкой?

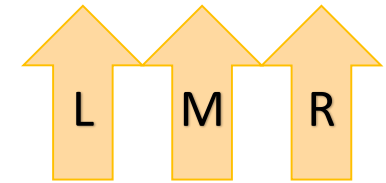
В бинарной сортировке вставками мы делим массив на два подмассива — отсортированный и несортированный. Первый элемент массива находится в отсортированном подмассиве, а остальные элементы — в несортированном.

Анализ временных комплексов:

Худший случай: $O(n^2)$

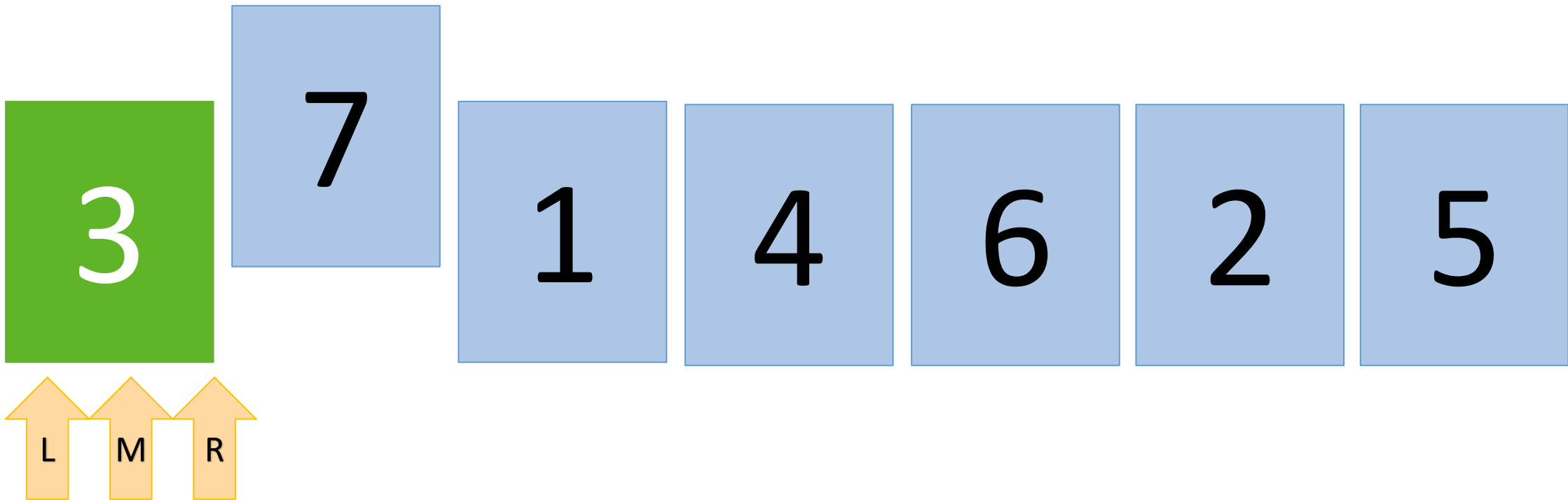
Лучший случай: $O(n \log n)$

Бинарная Сортировка Вставкой



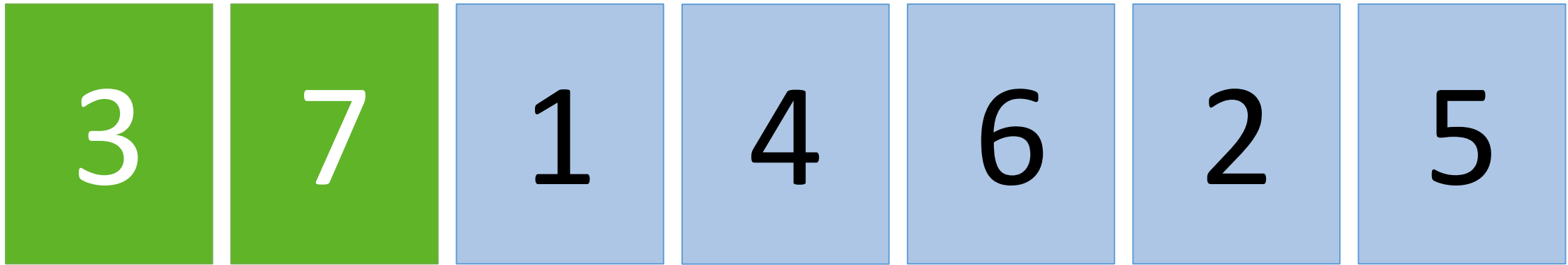
Мы считаем, что первый элемент отсортирован

Бинарная Сортировка Вставкой

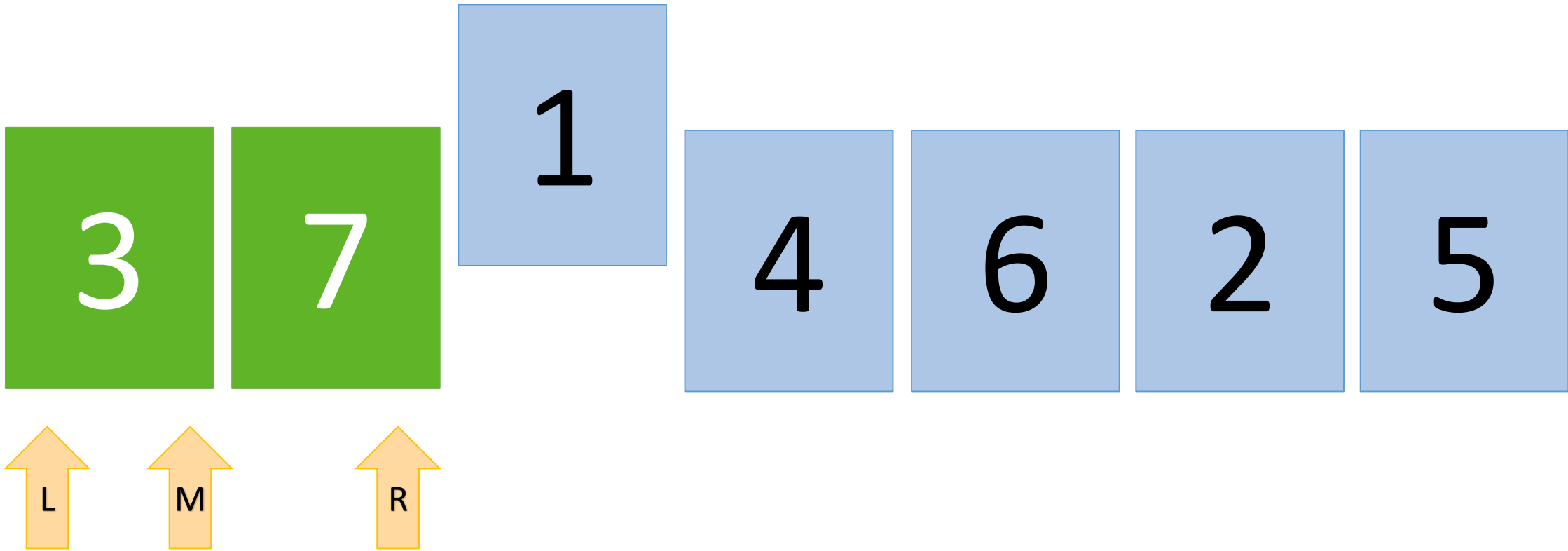


Выполняем бинарный поиск.
Мы сравниваем числа 3 и 7

Бинарная Сортировка Вставкой

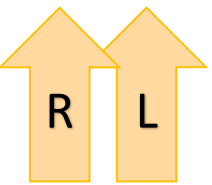
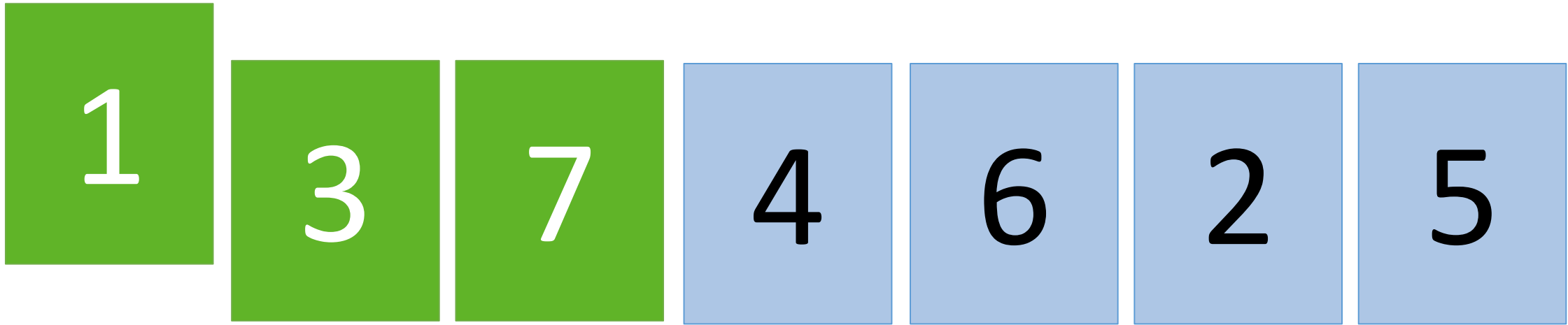


Бинарная Сортировка Вставкой



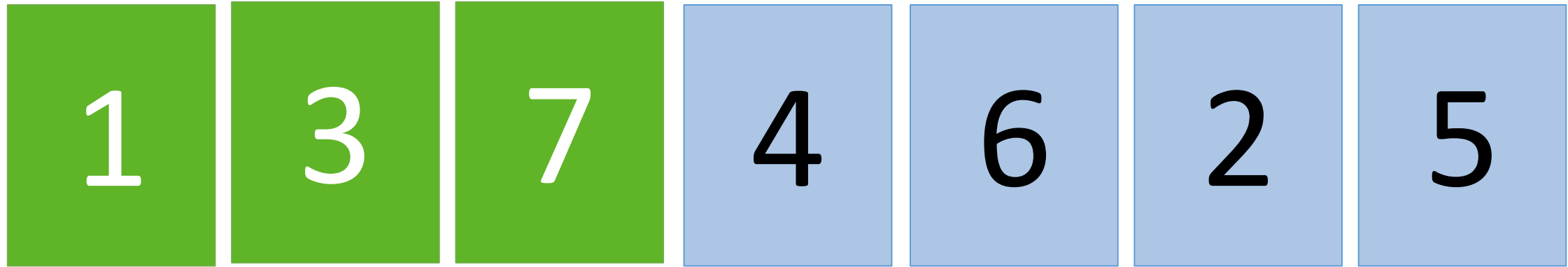
Мы выполняем бинарный поиск, 1 меньше 3 поэтому наш правый указатель идет влево.

Бинарная Сортировка Вставкой

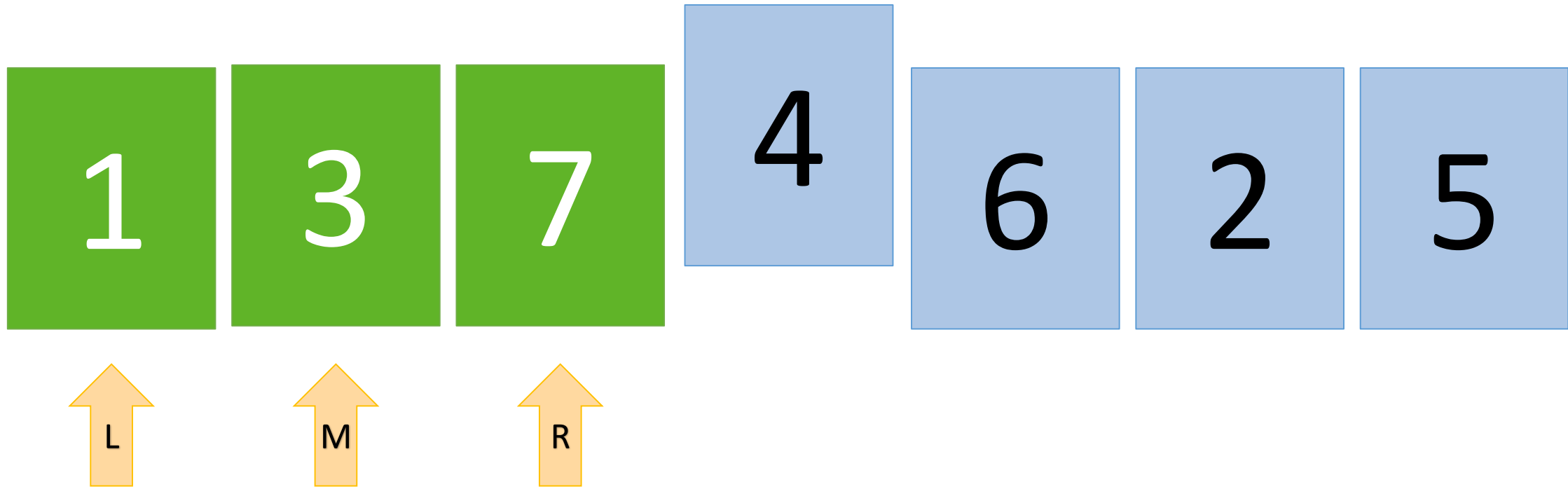


Мы выполняем бинарный поиск, 1 меньше 3 поэтому наш правый указатель идет влево.

Бинарная Сортировка Вставкой

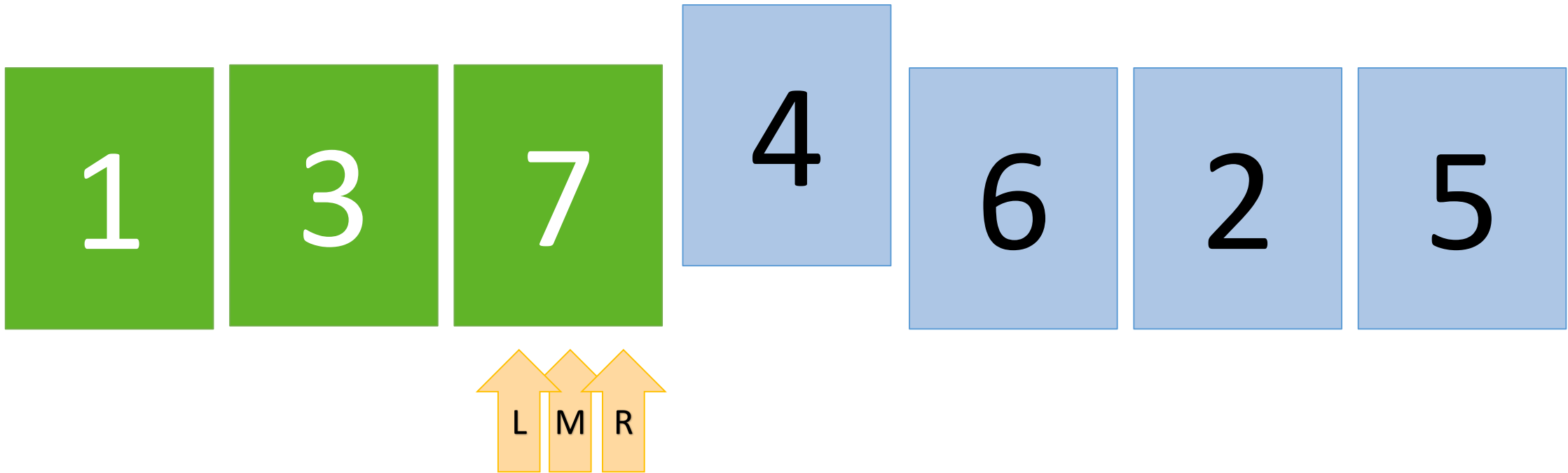


Бинарная Сортировка Вставкой



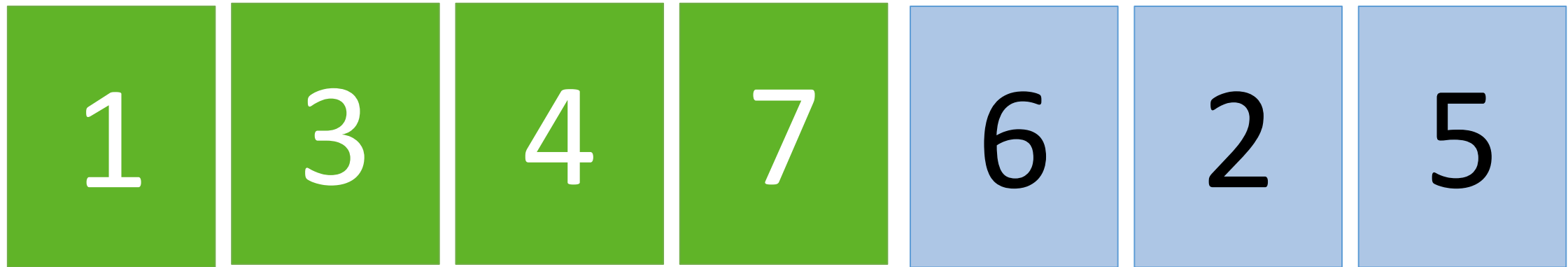
Мы выполняем бинарный поиск, 4 больше 3 поэтому наш правый указатель идет влево.

Бинарная Сортировка Вставкой

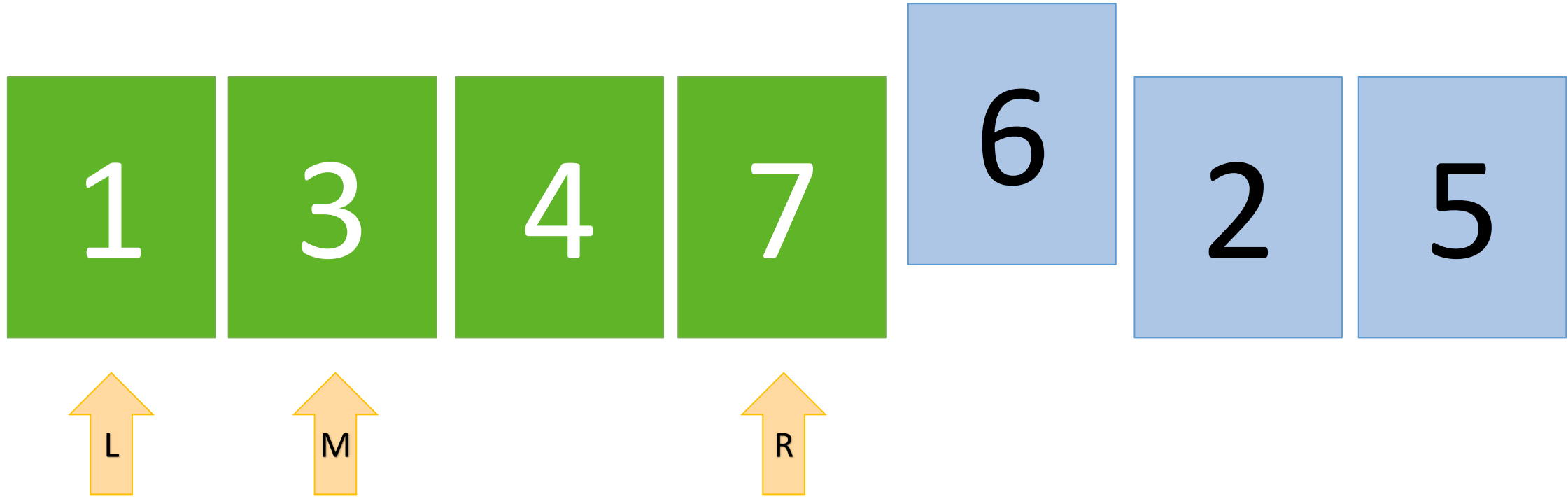


Мы делаем еще одно сравнение.

Бинарная Сортировка Вставкой

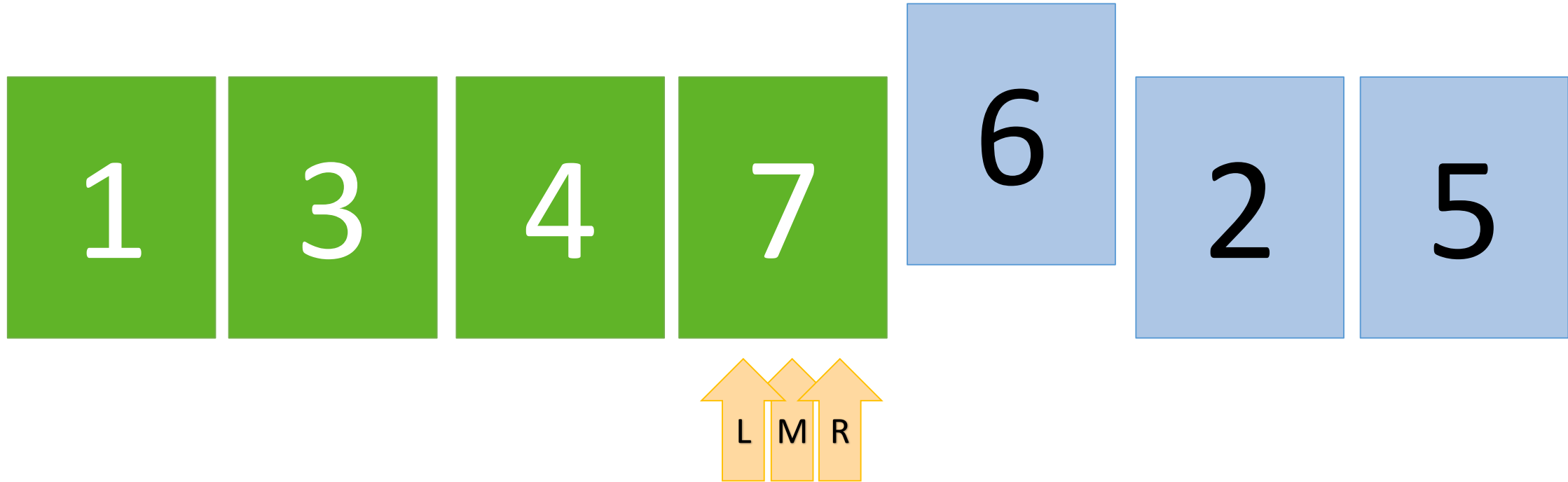


Бинарная Сортировка Вставкой



Выполняем бинарный поиск

Бинарная Сортировка Вставкой



Выполняем бинарный поиск
делаем еще одно сравнение

Бинарная Сортировка Вставкой



Бинарная Сортировка Вставкой



Выполняем бинарный поиск

Бинарная Сортировка Вставкой



Выполняем бинарный поиск

Бинарная Сортировка Вставкой

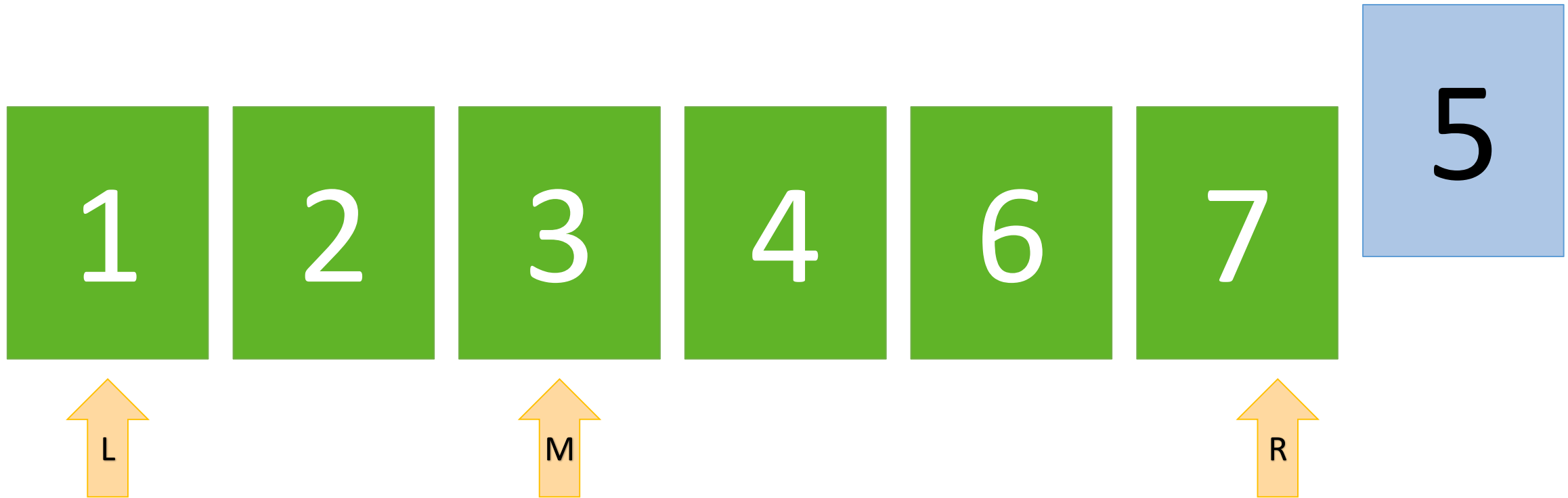


В итоге получается,
2 меньше 3 тогда

Бинарная Сортировка Вставкой

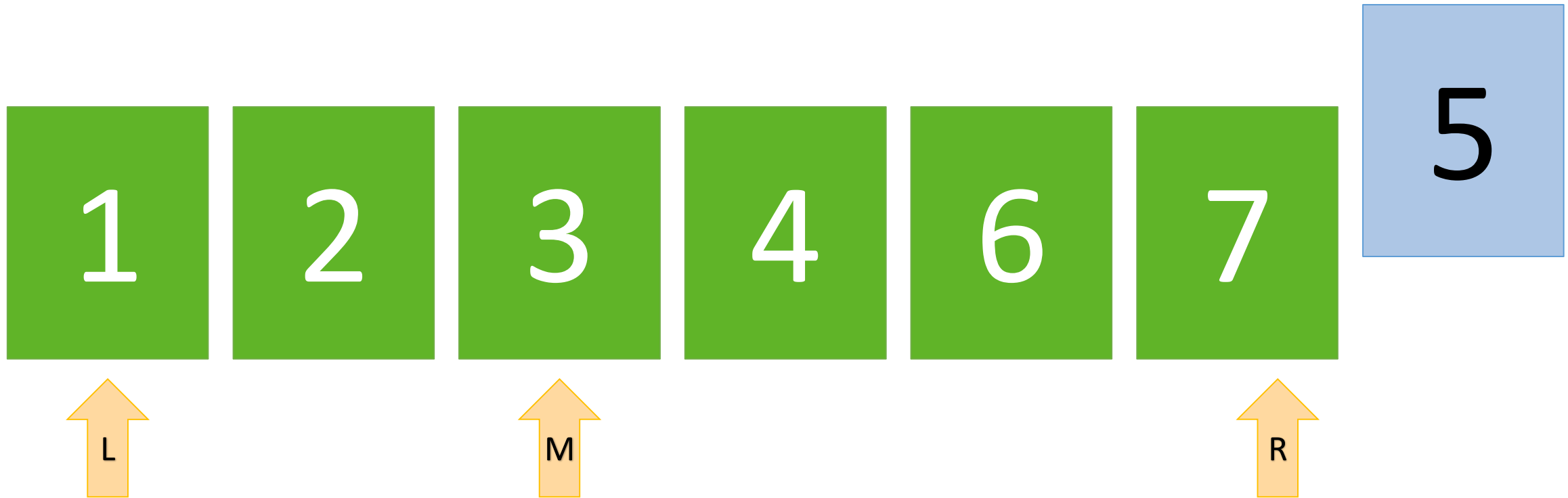


Бинарная Сортировка Вставкой



Выполняем бинарный поиск

Бинарная Сортировка Вставкой



Выполняем бинарный поиск

Бинарная Сортировка Вставкой



Выполняем бинарный поиск,
на этот раз мы сходимся на 4

Бинарная Сортировка Вставкой



5 больше 4, поэтому вставьте
в правую сторону.

```

int binarySearch(int a[], int item, int left, int right)
{
while (left <= right) {
    int mid = left + (right - left) / 2;
    if (item == a[mid])
        return mid + 1;
    else if (item > a[mid])
        left = mid + 1;
    else
        right = mid - 1;
}
    return left;
}

void insertionSort(int a[], int size)
{
    int loc, j, k, selected;
    for (int i = 1; i < size; ++i) {
        j = i - 1;
        selected = a[i];
        // find location where selected should be inserted
        loc = binarySearch(a, selected, 0, j);
        // Move all elements after location to create space
        while (j >= loc) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = selected;
    }
}

```

```

int binarySearch(int a[], int item, int left, int right)
{
    if (right <= left)
        return (item > a[left]) ? (left + 1) : left;
    int mid = (left + right) / 2;
    if (item == a[mid])
        return mid + 1;

    if (item > a[mid])
        return binarySearch(a, item, mid + 1, right);
    return binarySearch(a, item, left, mid - 1);
}

void insertionSort(int a[], int n)
{
    int loc, j, k, selected;
    for (int i = 1; i < n; ++i)
    {
        j = i - 1;
        selected = a[i];
        // find location where selected should be inserted
        loc = binarySearch(a, selected, 0, j);
        // Move all elements after location to create space
        while (j >= loc)
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = selected;
    }
}

```


Визуализация бинарной сортировки вставкой

<https://visualgo.net/en/sorting>